

## Practical-07

**Name:** S Rikhil

**Rollno:** 2520030480

**Section:** 8

### EXPERIMENT – Memory Address Space and /proc/<PID>/maps

#### Aim:

To write a C program that prints the addresses of **code, global, static, heap, and stack variables**, and analyse the Linux process address-space layout using /proc/<PID>/maps.

#### Objective:

- To understand the memory layout of a Linux process.
- To identify code, global, static, heap, and stack regions.
- To observe virtual memory mappings using /proc/<PID>/maps.
- To understand how Linux organizes memory for a running process.

#### Problem Statement:

Write a C program to display the addresses of different types of variables and use /proc/<PID>/maps to study the virtual memory organization of the running process.

#### Theory:

A Linux process has its own **virtual address space**. Important regions include:

| Memory Segment | Purpose                                        |
|----------------|------------------------------------------------|
| Code/Text      | Contains program instructions                  |
| Global/Data    | Contains initialized global variables          |
| BSS            | Contains uninitialized global/static variables |

| <b>Memory Segment</b>   | <b>Purpose</b>                                  |
|-------------------------|-------------------------------------------------|
| <b>Heap</b>             | Used for dynamically allocated memory           |
| <b>Stack</b>            | Stores local variables and function information |
| <b>Shared Libraries</b> | Libraries mapped into the process address space |

The `/proc/<PID>/maps` file shows the virtual memory regions currently mapped into a process.

Example:

55a1...-55a1... r-xp ... /program

55a1...-55a1... rw-p ... /program

7f2...-7f2... r-xp ... /libc.so.6

7ffd...-7ffd... rw-p ... [stack]

Here:

- r = Read
- w = Write
- x = Execute
- p = Private mapping

### **Algorithm:**

1. Declare a global variable.
2. Declare a static variable.
3. Create a local stack variable.
4. Create a function to represent the code segment.
5. Allocate memory dynamically using `malloc()`.
6. Print the addresses of all variables.
7. Print the process PID.

8. Keep the program running.
9. Open /proc/<PID>/maps from another terminal.
10. Analyse the virtual memory mappings.
11. identify the stack, heap, program, and shared-library regions.

### **C Program:**

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

int global_var = 10;

static int static_var = 20;

void code_function()

{

    printf("Code address   : %p\n", (void *)code_function);

}

int main()

{

    int stack_var = 30;

    int *heap_var;

    heap_var = (int *)malloc(sizeof(int));

    if (heap_var == NULL)

    {

        printf("Memory allocation failed\n");

        return 1;

    }

    *heap_var = 40;

    printf("Linux Process Memory Layout\n");
```

```

printf("-----\n");
printf("PID          : %d\n", getpid());
printf("Code address   : %p\n", (void *)code_function);
printf("Global address  : %p\n", (void *)&global_var);
printf("Static address   : %p\n", (void *)&static_var);
printf("Heap address    : %p\n", (void *)heap_var);
printf("Stack address   : %p\n", (void *)&stack_var);
printf("\nProcess is running...\n");
printf("Use another terminal to check /proc/%d/maps\n", getpid());
while (1)
{
    sleep(10);
} free(heap_var);
return 0;
}

```

## OUTPUT:

```

simra@simran315:~$ nano memory_layout.c
simra@simran315:~$ gcc memory_layout.c -o memory_layout
simra@simran315:~$ ./memory_layout
Linux Process Memory Layout
-----
PID          : 1123
Code address  : 0x5bd857c661e9
Global address : 0x5bd857c69010
Static address : 0x5bd857c69014
Heap address  : 0x5bd880bcb010
Stack address  : 0x7ffcb61fb0fc

Process is running...
Use another terminal to check /proc/1123/maps

```

```
simra@simran315:~$ cat /proc/1123/maps
5bd857c65000-5bd857c66000 r--p 00000000 08:30 42088
/home/simra/memory_layout
5bd857c66000-5bd857c67000 r-xp 00001000 08:30 42088
/home/simra/memory_layout
5bd857c67000-5bd857c68000 r--p 00002000 08:30 42088
/home/simra/memory_layout
5bd857c68000-5bd857c69000 r--p 00002000 08:30 42088
/home/simra/memory_layout
5bd857c69000-5bd857c6a000 rw-p 00003000 08:30 42088
/home/simra/memory_layout
5bd880bcb000-5bd880bec000 rw-p 00000000 00:00 0
[heap]
726ae1c00000-726ae1c28000 r--p 00000000 08:30 13647
/usr/lib/x86_64-linux-gnu/libc.so.6
726ae1c28000-726ae1dc0000 r-xp 00028000 08:30 13647
/usr/lib/x86_64-linux-gnu/libc.so.6
726ae1dc0000-726ae1e0e000 r--p 001c0000 08:30 13647
/usr/lib/x86_64-linux-gnu/libc.so.6
726ae1e0e000-726ae1e12000 r--p 0020d000 08:30 13647
/usr/lib/x86_64-linux-gnu/libc.so.6
726ae1e12000-726ae1e14000 rw-p 00211000 08:30 13647
/usr/lib/x86_64-linux-gnu/libc.so.6
726ae1e14000-726ae1e21000 rw-p 00000000 00:00 0
726ae1ece000-726ae1ed1000 rw-p 00000000 00:00 0
726ae1ed7000-726ae1ed9000 rw-p 00000000 00:00 0
726ae1ed9000-726ae1edd000 r--p 00000000 00:00 0
[vvar]
726ae1eddd000-726ae1edf000 r--p 00000000 00:00 0
[vvar_vclock]
726ae1edf000-726ae1ee1000 r-xp 00000000 00:00 0
[vdso]
726ae1ee1000-726ae1ee2000 r--p 00000000 08:30 13644
/usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
726ae1ee2000-726ae1f11000 r-xp 00001000 08:30 13644
/usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
726ae1f11000-726ae1f1c000 r--p 00030000 08:30 13644
```

```

726ae1f1f000-726ae1f20000 rw-p 00000000 00:00 0
7ffcb61dd000-7ffcb61fe000 rw-p 00000000 00:00 0
[stack]
simra@simran315:~$ pmap 1123
1123:    ./memory_layout
00005bd857c65000      4K r---- memory_layout
00005bd857c66000      4K r-x-- memory_layout
00005bd857c67000      4K r---- memory_layout
00005bd857c68000      4K r---- memory_layout
00005bd857c69000      4K rw--- memory_layout
00005bd880bcb000    132K rw--- [ anon ]
0000726ae1c00000    160K r---- libc.so.6
0000726ae1c28000   1632K r-x-- libc.so.6
0000726ae1dc0000    312K r---- libc.so.6
0000726ae1e0e000     16K r---- libc.so.6
0000726ae1e12000      8K rw--- libc.so.6
0000726ae1e14000     52K rw--- [ anon ]
0000726ae1ece000     12K rw--- [ anon ]
0000726ae1ed7000      8K rw--- [ anon ]
0000726ae1ed9000     16K r---- [ anon ]
0000726ae1edd000      8K r---- [ anon ]
0000726ae1edf000      8K r-x-- [ anon ]
0000726ae1ee1000      4K r---- ld-linux-x86-64.so.2
0000726ae1ee2000    188K r-x-- ld-linux-x86-64.so.2
0000726ae1f11000     44K r---- ld-linux-x86-64.so.2
0000726ae1f1c000      8K r---- ld-linux-x86-64.so.2
0000726ae1f1e000      4K rw--- ld-linux-x86-64.so.2
0000726ae1f1f000      4K rw--- [ anon ]
00007ffcb61dd000   132K rw--- [ stack ]
total                2768K

```

## Program Explanation:

### Code Segment

```
(void *)code_function
```

This displays the address of a function. Since program instructions are stored in the **code/text segment**, this represents the code region.

### Global Segment

```
int global_var = 10;
```

The global variable is stored in the process's global data area.

## Static Segment

```
static int static_var = 20;
```

The static variable has static storage duration and is stored in the program's data area.

## Heap

```
heap_var = (int *)malloc(sizeof(int));
```

malloc() dynamically allocates memory from the **heap**.

## Stack

```
int stack_var = 30;
```

A local variable inside main() is stored on the **stack**.

## Analysis:

- The r-xp mapping of the executable corresponds to the **code/text region**.
- The writable executable mapping contains **data/global/static areas**.
- [heap] represents dynamically allocated memory.
- [stack] represents the process stack.
- libc.so.6 represents a **shared library** mapped into the process.
- Each process sees these as **virtual addresses**, which Linux manages through virtual memory.

## Test Cases:

| Test Case | Command/Action     | Expected Result                    |
|-----------|--------------------|------------------------------------|
| 1         | Run program        | PID and memory addresses displayed |
| 2         | Check code address | Address of function displayed      |

| Test Case | Command/Action              | Expected Result                         |
|-----------|-----------------------------|-----------------------------------------|
| 3         | Check global/static address | Data-area addresses displayed           |
| 4         | Check heap address          | Dynamically allocated address displayed |
| 5         | Check stack address         | Local-variable address displayed        |
| 6         | cat /proc/PID/maps          | Virtual memory mappings displayed       |
| 7         | pmap PID                    | Process memory map displayed            |

#### Observation:

- The program displayed different virtual addresses for code, global, static, heap, and stack variables.
- The heap and stack appeared as separate regions in /proc/PID/maps.
- The executable and shared libraries were also mapped into the process address space.
- /proc/PID/maps provided information about the permissions and ranges of each virtual memory mapping.

#### Result:

The C program was successfully implemented to display addresses of the **code, global, static, heap, and stack** regions. The /proc/<PID>/maps file was used to analyse the virtual memory organization of the running process.

#### Conclusion:

The experiment demonstrated how Linux organizes a process into different virtual memory regions. The memory addresses printed by the program, together with /proc/<PID>/maps, helped identify the **code, data, heap, stack, and shared-library mappings**.